- Introduzione
  - Esame finale:
- Concetti fondamentali
  - qualità del software
  - Processi software
    - Metodi e strumenti
- Attività fondamentali dei processi software
  - Studio di fattibilità:
  - Code and fix
  - Acquisizione, analisi e specifica dei requisiti
  - Progettazione e Sviluppo
  - Verifica e Validazione
    - Test dei componenti
    - Test del sistema
    - Test del cliente
    - Test iterativo
  - Evoluzione
- Modelli di processi software
  - Modelli plan driven
    - Modello a cascata
      - Modello a cascata con retroazione
      - Modello a cascata a V
    - Modelli evolutivi
      - Modello prototipale
    - Modelli orientati al riuso
    - Modelli trasformazionali
  - Domande su elearning:
    - Domanda 1
    - Domanda 2
    - Domanda 3
  - Modelli agili
    - EXTREME PROGRAMMING
    - STORIE UTENTE
    - REFACTORING
    - sviluppo preceduto dai test
    - pair programming
  - Gestione agile della progettazione
    - SCRUM
- Tipologie di requisiti

- definizione dei requisiti
- requisiti funzionali e non funzionali
  - requisiti non funzionali

Introduzione

Nota:
appunti tratti dalle lezioni e dalle slide del corso di Ingegneria del Software, tenuto dal prof. Riccio Vincenzo

si inizia di solito alle 8:45
l'anno scorso tutti sono passati tra i vari appelli
materiale didattico su elearning - corso 725

Ricevimento:

- lunedì 15:00-17:00 - L2-02-BC (A2 129)
- volendo anche su Teams

esercitazioni in gruppo (2-3 persone), identiche a quelle dell'esame, si faranno a lezione e poi avverrà un confronto alla fine della lezione

libro: Ingegneria del software, Ian Sommerville, prima edizione

Esame finale:

- scritto carta e penna (120 minuti)
- 3 parti:
  - esercizi su analisi e modellazione
  - esercizi di testing
  - domande di teoria

tutte e 3 le parti devono risultare sufficienti per superare l'esame
orale solo per la lode

Concetti fondamentali

che cos'è il software?

- è un termine che indica l'insieme dei programmi per computer e la relativa **documentazione** (anche i documenti legali)

noi terremo conto anche di tutta la documentazione prodotta durante lo sviluppo del software, come modelli di progetto, manuali utente, siti web di supporto, ecc.

definizione IEEE:
"Il software è l'insieme di programmi, procedure, regole e documentazione relativi al funzionamento di un sistema di elaborazione dati"

"software is eating the world" - Marc Andreessen, 2011
si sta diffondendo in tutti i settori, non solo in informatica, con una capitalizzazione sempre più grande (ha superato i 1.468 miliardi)
con gli LLM non è cambiato il concetto, ma solo la velocità con cui il software viene sviluppato. Che ci piaccia o no, gli LLM rimangono software e "programmare" con essi cambia solo il livello del linguaggio usato per lo sviluppo (linguaggio naturale).

Software 1.0:
vedi il problema, pianifichi un algoritmo, progetti un algoritmo e implementi l'algoritmo
Software 2.0:
prepari i dati, definisci un modello, addestri il modello e valuti il modello
Software 3.0:
prepara i dati, definisci un modello, addestri il modello, valuti il modello - iterativamente (con un transformer)

software generico:

- software prodotto autonomamente da un'organizzazione per soddisfare le necessità di svariati clienti
- il produttore ha il controllo sulle specifiche del software

software custom:

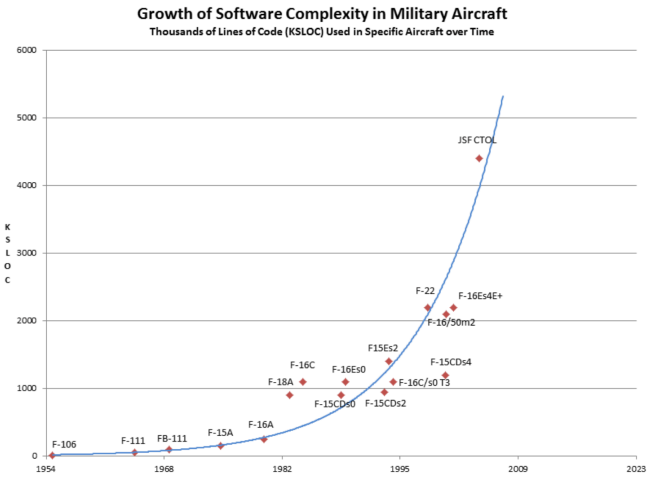
- software sviluppato da un'organizzazione su commissione di uno specifico cliente
- il produttore deve attenersi alle specifiche indicate dal cliente

qualità del software

è importante tenere conto della complessità, perché è fondamentale che un software sia **manutenibile**
Statisticamente, la manutenzione del codice è l'aspetto più costoso del ciclo di vita del software, quindi risolvere eventuali problemi durante la scrittura riduce drasticamente costi e tempi in fase di manutenzione.
se si scopre un problema:

- durante la scrittura: costo 1

- durante la rilettura: costo 7
- durante il testing: costo 20
- dopo la consegna: costo 100



Fonte: SAVI-AVSI (2023)

Il numero di linee di codice del software sta crescendo esponenzialmente.
Nonostante la maggiore complessità, la velocità con cui il software viene sviluppato è aumentata nel tempo, grazie sia a strumenti migliori per scriverlo, sia allo sviluppo di approcci di ingegneria del software.
Il successo o l'insuccesso di un progetto software è spesso legato all'applicazione dell'ingegneria del software per garantire la qualità del prodotto (anche se, nel breve termine, il codice potrebbe essere considerato "migliore").
Un codice facilmente manutenibile rende più veloce la scrittura di nuovo codice (e, a un certo punto, diventa impossibile scrivere nuovo codice senza questo processo).

Sviluppo personale	Sviluppo professionale
• Difficilmente il software sarà riusato in futuro da altri utenti • Non è necessario scrivere una guida, né un documento di progetto • I requisiti di qualità della documentazione sono meno stringenti	• Il software sarà usato da altre persone diverse dai propri sviluppatori • Sviluppo in team e, quindi, probabilmente il software sarà modificato e manutenuto da altri sviluppatori • Il software deve possedere caratteristiche di qualità

Le caratteristiche qualitative di un software professionale sono:

- **Accettabilità**
- **Fidatezza e Protezione**
- **Efficienza**
- **Mantenibilità**

L'insieme specifico delle caratteristiche qualitative di un sistema software dipende dalla sua applicazione.

Sviluppare software sta diventando sempre più costoso, perché le operazioni che un software deve compiere sono sempre più complesse (siamo passati dal giocare a Pong al creare macchine a guida autonoma).
Questo fenomeno si cercò di risolvere nel 1968, alla NATO Software Engineering Conference di Garmisch, dove il problema venne definito come "**software crisis**".
La soluzione fu l'**ingegneria del software**, che poi portò alla nascita dell'**IEEE** (Institute of Electrical and Electronics Engineers), ovvero all'applicazione dei principi dell'ingegneria ai processi di sviluppo del software.

"L'Ingegneria del Software
è una disciplina
ingegneristica che
riguarda tutti gli aspetti
della produzione del software"

L'Ingegneria del Software (IS) è una disciplina ingegneristica (è l'intersezione tra ingegneria e informatica).

Processi software

Processo software: insieme di attività che porta alla creazione o all'evoluzione di un prodotto software.

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione

Metodi e strumenti

Metodi:

- Approcci strutturati per sviluppare software di qualità, a costi contenuti ed entro i tempi di consegna
- Tali metodi forniscono una guida alle attività dei processi e alla relativa organizzazione

Non sono comandamenti, ma solide linee guida sviluppate nel corso di decenni.

Strumenti:

- Sistemi software usati per aiutare le attività dei processi software (ad es., analisi, modellazione, debugging, testing)

ALCUNE SFIDE PER I PROCESSI SOFTWARE:

- Diversità
- Consegna
- Fiducia
  - creare documentazione, certificazioni, ecc. per far sì che l'utente (o un altro programmatore) possa fidarsi del software;
  - ci sono enti e standard che certificano la qualità del software;
- Scala
  - avere un software che può essere scalato facilmente qualora il numero di utenti aumenti

PRINCIPI FONDAMENTALI:

- I metodi specifici, le tecniche e gli strumenti utilizzati dipendono dall’organizzazione che sviluppa il software, dal tipo di software e dalle persone coinvolte nel processo di sviluppo
- Non ci sono metodi di ingegneria del software universali applicabili a tutti i sistemi e a tutte le aziende.
- I concetti fondamentali dell’ingegneria del software sono **indipendenti dal linguaggio di programmazione** utilizzato per sviluppare il software

Il **processo** va sempre chiarito sin da subito, soprattutto quando si lavora in gruppo; altrimenti l'organizzazione e l'efficienza del gruppo collassano. Si devono definire sin da subito i ruoli, le responsabilità e le attività di ogni membro del gruppo; altrimenti il progetto fallisce.

FIDATEZZA E PRESTAZIONI:
il software deve fare quello che deve fare, in maniera efficiente (e senza spreco di risorse), e in modo sicuro

REQUISITI:

- È importante capire e gestire la specifica e i requisiti del software

- Occorre sapere che cosa si aspettano clienti e utenti del sistema e occorre gestire le loro aspettative in modo che possa essere rilasciato un sistema utile entro i costi e i tempi previsti

RIUSO:
Riutilizzare vecchio codice è spesso più rapido ed economico di scrivere nuovo codice da zero, ma bisogna stare attenti che il codice riutilizzato non abbia bug o sia effettivamente adatto al nuovo contesto.

MODELLO DI PROCESSO:

- Ciascuna organizzazione utilizza il proprio processo
- Il modello di processo software è una rappresentazione semplificata e astratta che descrive l'intero ciclo di vita del software

"come Platone e il mondo delle idee"

Attività fondamentali dei processi software

- Ciascun processo è composto da attività fondamentali
- Tutti i processi software condividono queste attività fondamentali
- Tali attività possono essere organizzate e realizzate in modi diversi in processi diversi

4 ATTIVITÀ FONDAMENTALI DEI PROCESSI SOFTWARE:

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione

Studio di fattibilità:

Stabilisce se lo sviluppo debba essere avviato, ossia:

- Se esiste un mercato per il software
- Se il software sia tecnicamente ed economicamente realistico

Definisce quali sono le alternative possibili e le scelte più ragionevoli, stimando le risorse (finanziarie e umane) necessarie per ciascuna alternativa.

Se in un progetto ci sono elementi già sviluppati da altri (per esempio open source), ciò permette di risparmiare tempo e denaro; se invece l'intero progetto è già stato sviluppato a un

prezzo minore di quello che potremmo proporre, spesso è il caso di abbandonare il progetto.

Fornisce un report di fattibilità con:

- Definizione del problema
- Valutazione Costi/Benefici
- Risorse finanziarie e umane
- Soluzioni alternative
- Tempi di consegna e modalità di sviluppo

Tale studio dovrebbe essere **relativamente rapido** e **poco costoso** (se sto un mese a fare uno studio sui telefoni cellulari per capire quale acquistare e nel mentre esce una nuova generazione, ho buttato il mio tempo perché devo ricominciare da capo)

Code and fix

Approccio alla produzione di software che consiste nello:

- Scrivere codice
- Aggiustare il codice per correggere errori, migliorare e/o aggiungere funzionalità

Correggere gli errori (anche usando gli LLM per fare il debugging), uno alla volta, non è l'approccio ottimale per progetti di grandi dimensioni, motivo per cui sono necessari approcci più organizzati allo sviluppo del software.

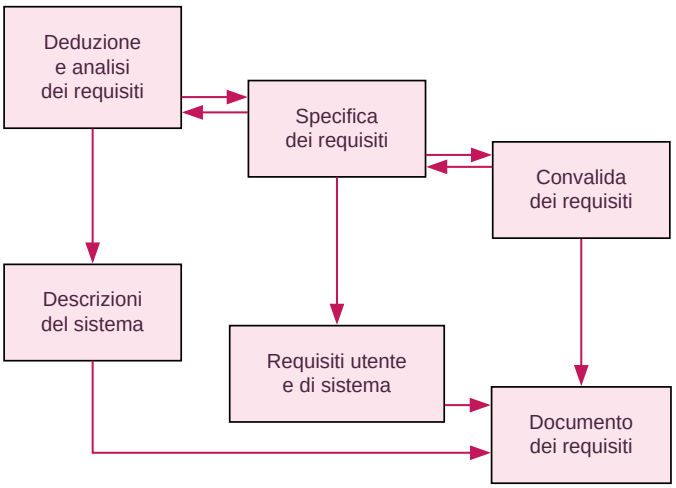
Acquisizione, analisi e specifica dei requisiti

- Attività per stabilire **cosa** il software dovrà fare
- Specificare funzionalità e qualità
- Definisce, tramite l'interazione con il committente, funzioni, vincoli, prestazioni, interfacce...
- Attività critica

Bisogna dedurre e analizzare i requisiti del software non solo dal cliente, ma anche dai **portatori di interesse** (stakeholder):

- clienti
- committenti
- enti amministrativi esterni

Bisogna avere un documento formale con cui descrivere in dettaglio le funzionalità e i vincoli del progetto (che poi verranno convalidati da diversi enti).



La **deduzione** richiede spirito critico e può coinvolgere:

- osservazione di sistemi già esistenti
- discussione con i possibili utenti

La **specifica** traduce le informazioni dedotte in un insieme di requisiti:

- Requisiti di **sistema**
- Requisiti **utente**

La **convalida** controlla che i requisiti siano realistici, coerenti e completi.
Durante questo possono essere rilevati errori nel documento dei requisiti e in tal caso il documento dei requisiti deve essere modificato e corretto

Al termine di questo processo, se il progetto rispecchia gli standard, viene approvato.

Tutte queste attività sono interconnesse, perché in ogni fase si possono notare dei difetti o errori avvenuti in una fase precedente

Progettazione e Sviluppo

Divisa in due fasi:

- **progettazione**
- **sviluppo**

Le attività della fase di progettazione sono interconnesse e interdipendenti
I progettisti non ottengono immediatamente un risultato completo
Le nuove informazioni sul progetto influenzano le precedenti scelte progettuali

Progettazione dell'architettura:

- definire classi, strutture, ecc.

Progettare un'interfaccia:

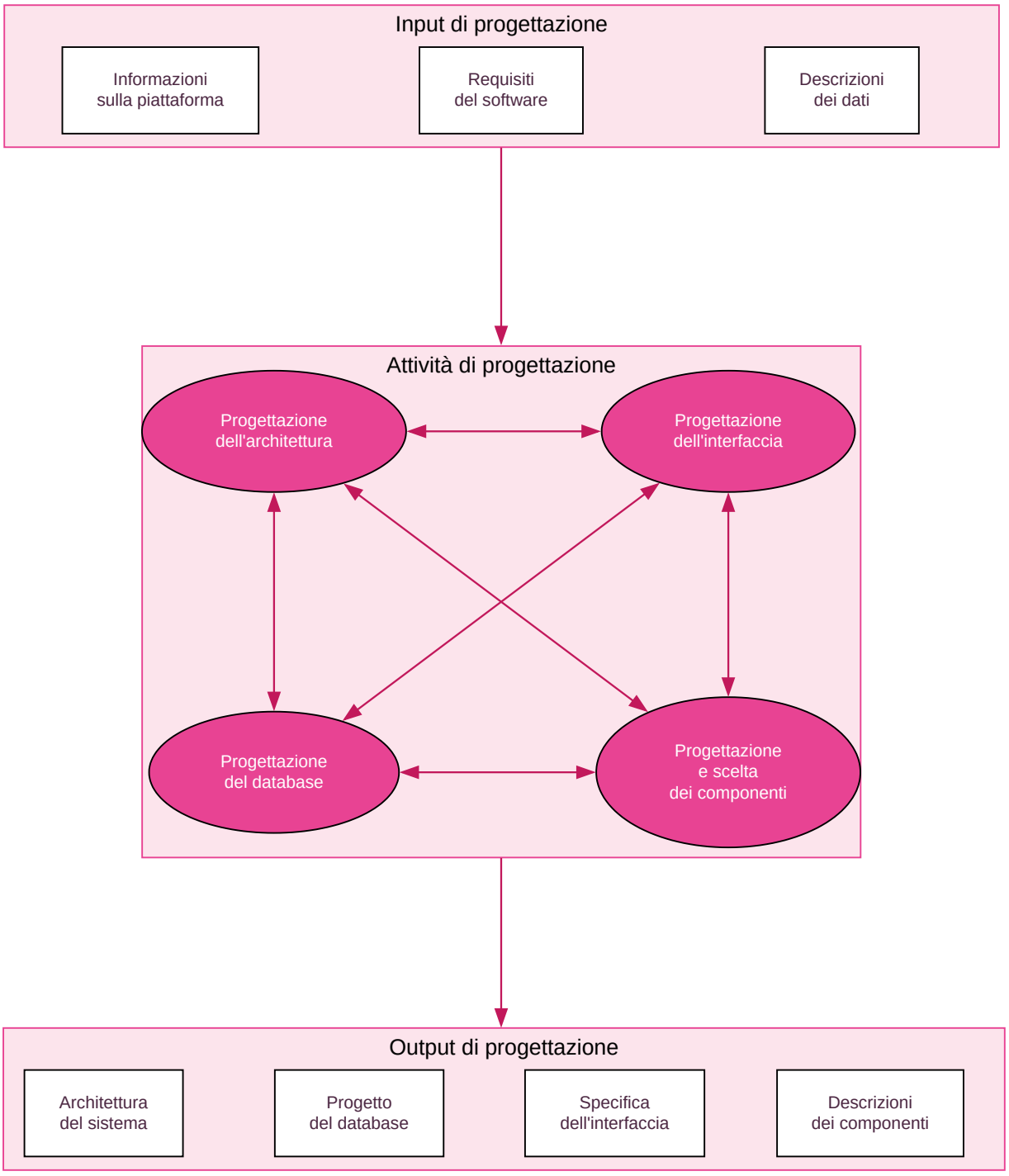
- non deve essere necessariamente un'interfaccia grafica, ma deve essere in grado di soddisfare tutte le interazioni necessarie con l'esterno;
- una volta definita un'interfaccia, posso espandere il software perché è già suddiviso in moduli.

Scelta dei componenti:

- avere componenti riutilizzabili e adattarli
- progettare componenti riutilizzabili
- bisogna mantenere i componenti flessibili, in modo che anche un altro sviluppatore possa riutilizzarli in un contesto diverso

L'output finale di queste fasi è un progetto software (formale o informale) che descrive:

- La struttura del software che si deve implementare
- I modelli e le strutture di dati usati dal sistema
- Le interfacce tra i componenti del sistema



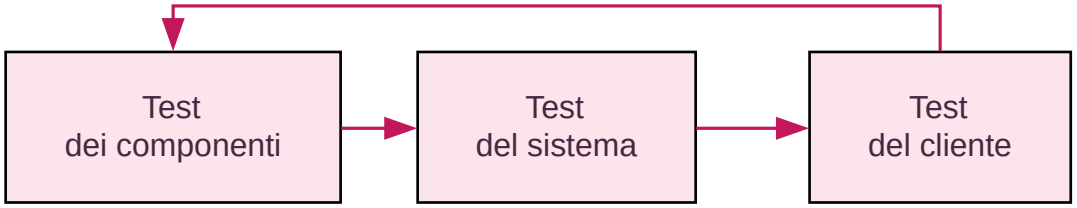
La parte di **sviluppo** è la meno importante in questo corso, perché è fortemente dipendente dalle restrizioni o indicazioni di una determinata azienda, cliente, ecc.

Verifica e Validazione

verifica: mostrare che un sistema è conforme alle sue specifiche
validazione: mostrare che un sistema soddisfa le esigenze del cliente

Queste due fasi sono complementari tra loro, perché posso avere un ottimo software, ma, se al cliente non piace, non è un buon software (o se non rispecchia determinate normative).
ogni volta che convoco clienti/stakeholder per la validazione, è un processo costoso, quindi è importante arrivare alla validazione con un software funzionante e che rispetti le aspettative.

per questo motivo abbiamo il **testing**:
a un software funzionante forniamo dei dati di prova (realistici) e si verifica che il software non fallisca e che i risultati siano conformi alle attese.



Test dei componenti

il componente può essere una funzione, una classe, un microservizio, ecc.
dobbiamo iniziare con il test del singolo componente (il fatto che tutti i componenti singolarmente funzionino non significa che il sistema complessivo funzioni)

un esempio di un sistema che funziona anche se un singolo componente non funziona:
una fotocamera che deve determinare la traiettoria di un oggetto in movimento, se un singolo componente è alterato (magari per una goccia di pioggia sul sensore), il modello del sensore fornisce un dato, ma il sistema, basandosi sul contesto e sugli altri sensori/dati, è in grado di correggere l'errore e fornire comunque un risultato corretto.

Test del sistema

una volta testati tutti i singoli componenti, si passa al test del sistema completo.
come accennato prima, se alcuni componenti funzionano bene, ma non sono stati scritti con interfacce compatibili (o altri problemi), il sistema nel complesso non funziona.
bisogna quindi fare del debugging e capire dove si trova il problema

Test del cliente

il sistema viene testato con i dati reali di un cliente in condizioni realistiche.
questo test può rilevare problemi con i requisiti
ci sono dei margini di tolleranza

Test iterativo

Alcuni errori nei componenti del programma possono apparire durante il test del sistema, oppure il cliente può stimolare il sistema in modi non previsti dai dati simulati.

Evoluzione

solitamente non bisognerebbe mai reinventare la ruota, ma è importante usare sistemi esistenti (e già testati esaustivamente) e modificarli per le proprie esigenze.
Con l'uso intensivo di LLM, si tende però a riscrivere librerie già esistenti da zero, perché la velocità con cui un LLM produce codice è paragonabile al cercare una libreria già esistente e adattarla alle proprie esigenze.

Nel corso della sua vita, il software viene modificato continuamente:

- manutenzione correttiva
- migliorie (efficienza o funzionalità)
- adattamento a nuovi mutamenti del contesto

Modelli di processi software

DEFINIZIONE del modello del ciclo di vita del software (CVS):
Un modello del ciclo di vita del software è una caratterizzazione descrittiva o prescrittiva di come un sistema software viene o dovrebbe essere sviluppato

- (W. Scacchi in Encyclopedia of Software Engineering)

DEFINIZIONE STANDARD ISO/IEC/IEEE 12207:2026:
Software Life Cycle:
evolution of a system, product, service, project or other human-made entity from conception through retirement

Life Cycle Model:
framework of processes and activities concerned with the life cycle which can be organized into stages, acting as a common reference for communication and understanding

Nota: viene ancora definito **human-made entity** perché, anche se un LLM ha scritto il codice o ha contribuito alla fase di progettazione, è comunque un essere umano a dover validare il

software e a doverlo rilasciare; quindi il software è sempre un prodotto umano.

Non esiste un processo universale.

Il modello astrae il processo del ciclo di vita reale di un software.

nel '96 la definizione era molto più complessa:
non spiega neanche effettivamente a cosa servisse il modello ed aveva più righe di quella attuale

Ogni processo ha diverse possibili descrizioni:

- **Architetturale** – descrive la sequenza di attività senza fornire dettagli sulle specifiche attività (utilizzato in questo corso);
- **Data-flow** – evidenzia le trasformazioni dei dati operate dalle attività del processo;
- **Role/action** – I ruoli delle persone coinvolte nel processo e le relative responsabilità

Perché si parla di ruoli e non di persone?
una persona può avere più ruoli, o un singolo ruolo può essere ricoperto da più persone.

La **descrizione** del processo software include:

- Attività
- prodotti
- ruoli
- pre- e post-condizioni

Quale modello devo scegliere?

- **plan driven** (tutto deve essere pianificato in anticipo, e il processo deve seguire il piano)
- **agili** (non fortemente guidati da un piano, ma si adattano velocemente ai cambiamenti)

Queste sono classi di modelli e bisogna sapere quali applicare in base al contesto; talvolta è anche solo una preferenza personale.

nota: nel PW3 da pagina 13 ci sono un sacco di domande di teoria che spesso chiedono all'esame

modelli che vedremo in questo corso:

- modello a cascata
- modelli evolutivi
- modelli orientati al riuso
- modelli trasformazionali (cenni)

(sono tutti processi guidati da un piano, quindi plan driven)

Modelli plan driven

Modello a cascata

Mettiamo le varie attività in sequenza; la maggior parte delle attività, degli artefatti prodotti in ogni fase e delle **milestone** è definita sin dall'inizio del progetto.
le fasi riflettono direttamente le attività fondamentali dei processi software:

- Definizione dei requisiti (produce lista dei requisiti)
- Progettazione del sistema e del software
- Implementazione e test delle unità (produce codice di test)
- Integrazione e test del sistema (produce test di sistema)
- operatività e manutenzione (sw finito integrato e testato)

la **documentazione** è fondamentale in questo modello, prodotto alla fine di ogni fase, ogni documento deve essere approvato prima di passare alla fase successiva.

Se ho commesso degli errori nei requisiti, se ne pagheranno le conseguenze molto tardi (i vari passaggi non possono essere modificati facilmente): non si può tornare indietro, perché il modello a cascata è **rigido**.
vantaggi:

- le fasi sono ben definite (non ci sono sorprese)
- gli output di ciascuna fase sono precisamente individuati

svantaggi (siamo nel mondo reale):

- essere così rigidi non è sempre fattibile (spesso il cliente stesso non sa cosa vuole)
- produce troppa documentazione (ora con gli LLM è più semplice, ma rimane comunque un processo lungo e costoso)
- poco flessibile: se succede un imprevisto bisogna tornare all'inizio e rifare tutto il processo

Ma perché si usa ancora il modello a cascata?
se ci sono degli enti che devono approvare il software, è necessario avere una documentazione completa e dettagliata (per esempio un sw che gestisce un aereo)

quando non usarlo?

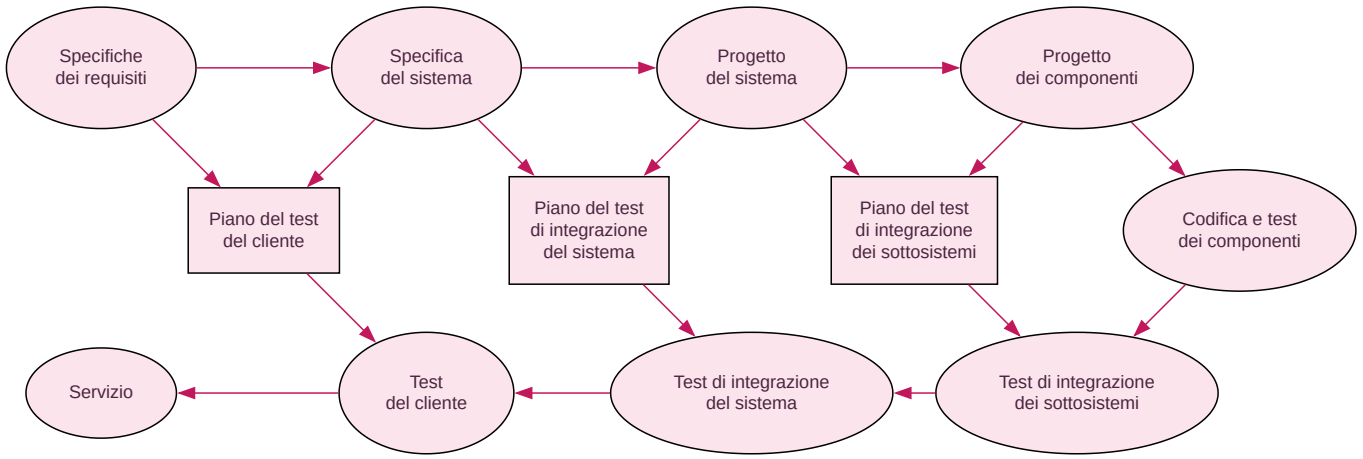
- con piccoli team di sviluppo
- con progetti non critici

Modello a cascata con retroazione

Nel modello, a ogni passaggio di fase, si esegue anche uno scambio di opinioni e feedback, rendendo possibile tornare indietro e correggere eventuali errori nella fase precedente.
ogni volta che si torna indietro, la documentazione precedente deve essere aggiornata (il processo rimane estremamente formale)
il salto si può fare solo tra fasi adiacenti, ma si può tornare indietro più volte (Sempre aggiornando la documentazione)

Modello a cascata a V

è un modello molto più utilizzabile nel mondo reale:
per ogni fase di progettazione possiamo già andare a vedere con il cliente se i requisiti sono corretti o sono stati interpretati correttamente, così per ogni fase di sviluppo vengono realizzati dei test.



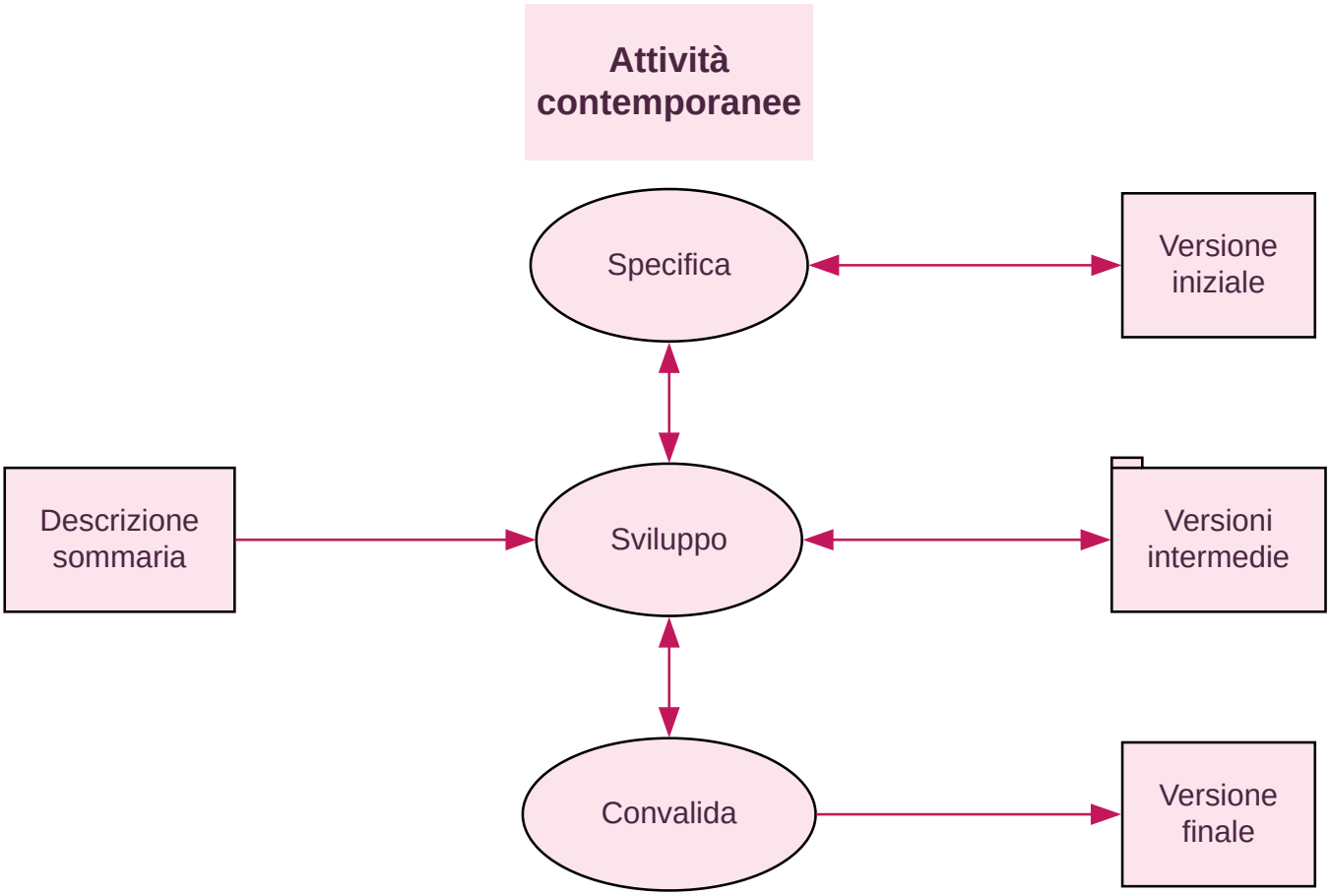
nei progetti grossi, il team di sviluppo è diverso dal team di testing, il team di testing deve cercare di fare "fallire" il sw o il piano in ogni modo possibile in ogni fase, così da poter correggere eventuali errori prima di passare alla fase successiva.

tutto deve essere sempre documentato ad ogni passaggio

Modelli evolutivi

- Modello a sviluppo/consegna incrementale
  - lavorare con il cliente
- Modello prototipale
  - sviluppo di un prototipo per comprendere i requisiti del cliente

Si inizia con un'idea di base e si aggiungono elementi nel tempo, iniziando con requisiti relativamente vaghi, per poi arrivare a un prodotto finito e completo.



modello a sviluppo incrementale:
questo modello consente di avere feedback immediati e continui dal cliente:
Al cliente non vengono forniti documenti (o ne vengono forniti pochi), ma solamente "esempi pratici" del software; in questa maniera, se al cliente non piace qualcosa, si può modificare il progetto senza dover rifare tutto il processo dall'inizio.

dopo diverse versioni intermedie, si arriva ad una versione finale, ed è quella che rilascerò al cliente

modello a consegna incrementale:
Per essere sicuri che il software funzioni esattamente come dovrebbe, vado a installarlo nell'ambiente operativo del cliente man mano che procede lo sviluppo.

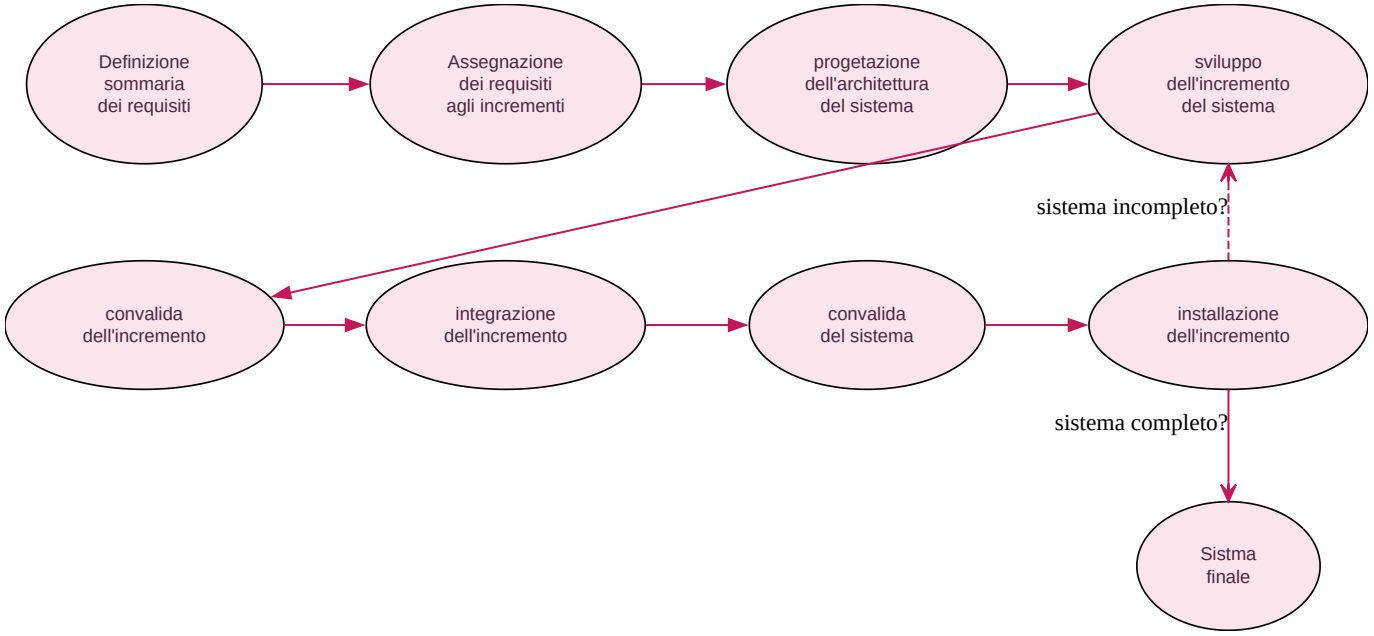
vantaggi:

- il cliente può usare il software nell'ambiente operativo reale
- feedback molto realistico e immediato

svantaggi:

- bisogna avere un cliente disponibile a installare la demo sui propri sistemi e che abbia abbastanza tempo per testarla
- richiede parecchio tempo

ogni incremento rilascia delle priorità da implementare (definite con il cliente), una volta approvate vengono "congelate" e non si modificano più.
vado quindi a **mappare** i requisiti in base alla priorità:



sviluppo incrementale	consegna incrementale
la valutazione della prima versione è effettuata da un proxy degli utenti finali in un ambiente operativo diverso da quello target. Le versioni intermedie non sono solitamente rilasciate al cliente	permette una valutazione più realistica e rappresentativa dell'utilizzo reale del software perché ciascun incremento può essere rilasciato agli utenti finali nell'ambiente operativo del sistema

vantaggi:

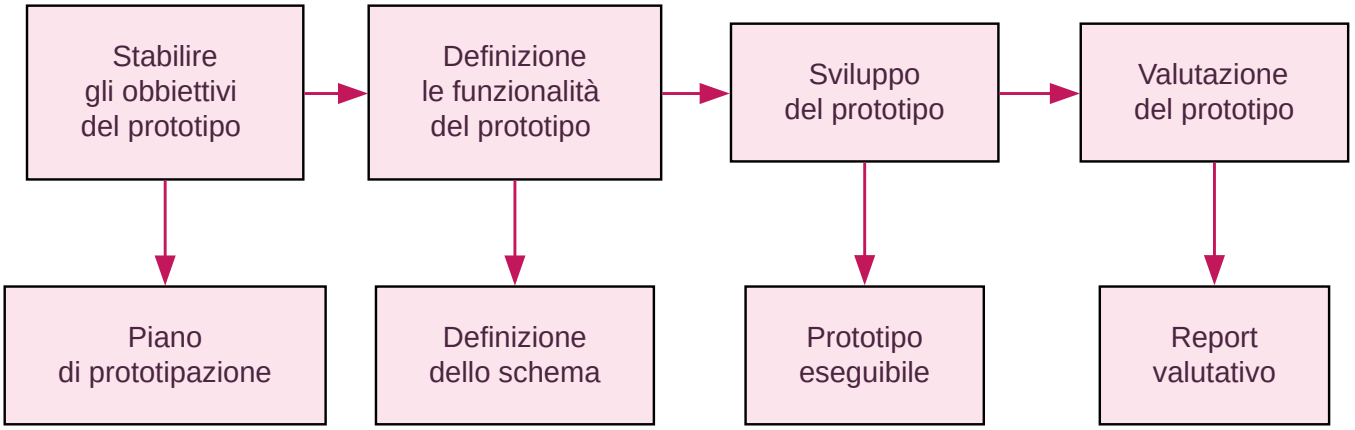
- i problemi si trovano più rapidamente
- le funzionalità critiche, essendo state rilasciate subito, possono essere testate estensivamente (e hanno più feedback)

problemi:

- poca documentazione (manca visibilità del processo)
- difficile da mantenere per i continui cambiamenti
- codice spesso mal strutturato

Modello prototipale

Il prototipo DEVE essere usa e getta perché, per definizione, non è perfetto; quindi, se usato per il prodotto finale, molto probabilmente si porterà dietro un sacco di problemi e bug.



Il processo avviene sempre con uno schema rigido, perché vogliamo spendere il meno possibile (realizzare prototipi non è gratis) e il più velocemente possibile.
Il grande vantaggio del prototipo è che posso avere un feedback molto più realistico e immediato, perché il cliente può interagire con il prototipo e capire se è quello che vuole o meno.
Il problema della valutazione del prototipo è che le persone che lo valutano devono avere chiari gli obiettivi del progetto, perché si rischia di ricevere un feedback non utile o fuorviante.

Uno dei problemi di questo modello può essere:

- rappresentabilità del prototipo (non è detto che il prototipo sia rappresentativo del prodotto finale)
- i tester devono essere variegati (e non con eventuali conflitti di interesse)
- l'interazione dell'utente con il prototipo potrebbe non essere rappresentativa di quella con il prodotto finale

possiamo piazzare una fase di prototipazione durante la fase di **valutazione dei requisiti**, in modo da avere una valutazione più realistica e migliorare la comprensione dei requisiti da parte del cliente.

Ma quale linguaggio usare per un progetto?
durante la fase di progettazione in un modello a cascata, si sceglie il linguaggio di programmazione da usare per lo sviluppo del software

Back to back testing

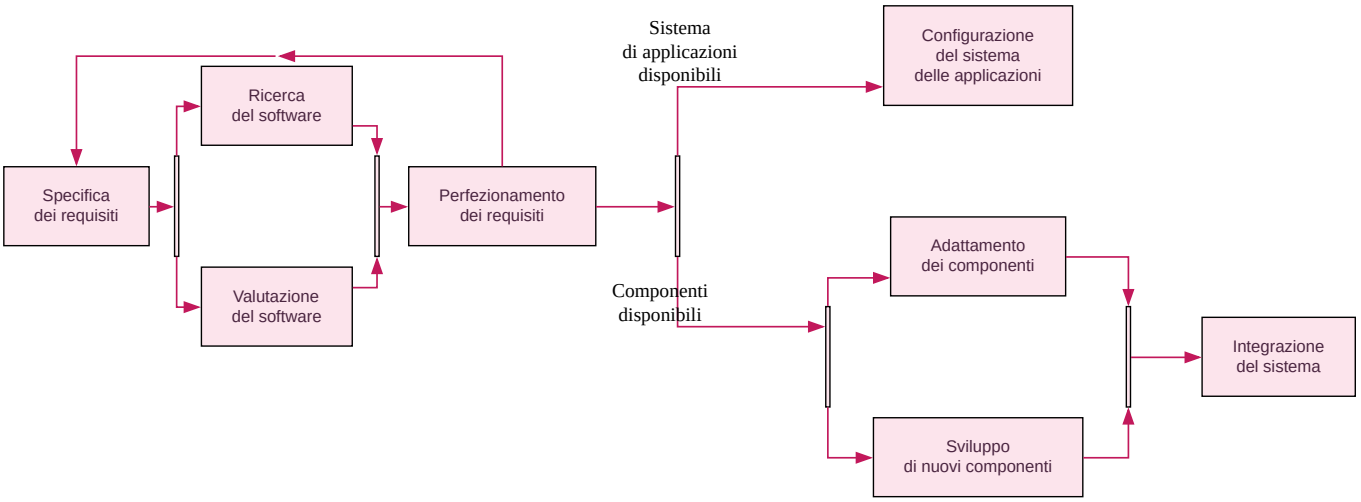
Abbiamo un prototipo finale (ma rimane ancora un prototipo) e lo usiamo nella fase di testing come "oracolo", per avere una base di confronto per il prodotto finale.

Modelli orientati al riuso

"non reinventare la ruota"
un buon progetto software dovrebbe essere costruito con componenti già esistenti, in modo da risparmiare tempo e denaro.

ma cosa riuso?
in maniera sistematica:

- componenti riutilizzabili (librerie, package, framework, ecc.)
- interi sistemi (COTS - Commercial Off-The-Shelf Software: componenti commerciali che possono essere acquistati e integrati nel progetto)



vantaggi:

- riduce la quantità di codice da scrivere
- riduce i costi di sviluppo
- riduce i rischi
- maggiore velocità di sviluppo

svantaggi:

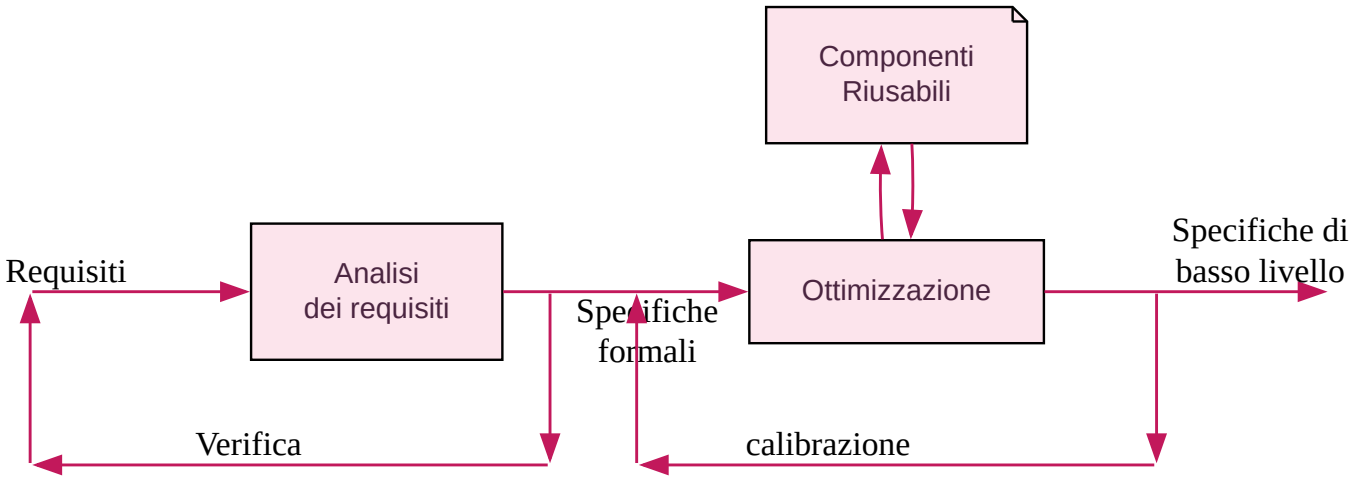
- compromessi nei requisiti (non sempre il componente riutilizzato soddisfa tutti i requisiti)
- evoluzione dei componenti riutilizzati (se un componente viene aggiornato, il mio software potrebbe non funzionare più)

Modelli trasformazionali

specifiche definite attraverso linguaggi formali

- specifiche algebriche
- modelli di stato

con le trasformazioni formalmente definite (formali) posso preservare proprietà già dimostrate



- la fase di **verifica** è molto spesso automatizzata, quindi rilevo e evito errori o incoerenze già prima della progettazione
- la fase di calibrazione è una fase di ottimizzazione, dove ad ogni passaggio cerco di scendere di livello

vantaggi

- specifiche più precise
- possibilità di verificare proprietà prima dellimplementazione
- forte garanzie su proprietà critiche

Problemi

- uso intensivo di liunguaggi formali
- difficile specificare formalmente sistemi complessi
- difficoltà di convalidare i requisiti

Applicabilità

- usati per parti critiche, farlo per l'intero sistema spesso è troppo costoso e complesso

Domande su elearning:

Domanda 1

Un team deve realizzare un sistema gestionale aziendale.
Il progetto deve essere completato rapidamente e con budget limitato, sfruttando moduli e

componenti già disponibili.
Quale modello è più efficace?

- Modello a consegna incrementale
- Modello a cascata
- **Modello orientato al riuso**
- Modello prototipale

Domanda 2

Un’azienda aerospaziale deve sviluppare il software di controllo di un satellite.
I requisiti sono completi e stabili, e ogni modifica deve essere rigorosamente tracciata e approvata prima dell’implementazione.
Quale modello di processo è più appropriato?

- **Modello a cascata**
- Modello a sviluppo incrementale
- Modello prototipale
- Modello a consegna incrementale

Domanda 3

Un’azienda sta sviluppando un nuovo sistema di prenotazione per un’agenzia di viaggi online.
Il progetto è pianificato in tre incrementi di 4 settimane.
Nei primi due incrementi, il team lavora su:

- l’integrazione con i database delle compagnie aeree;
- la gestione delle transazioni e dei pagamenti.

Solo al terzo incremento i committenti potranno accedere al sistema tramite un’interfaccia web per effettuare prenotazioni reali.
Il cliente riceve aggiornamenti tecnici ma non un prodotto utilizzabile fino al terzo rilascio, quando il software verrà provato nell'ambiente operativo aziendale.
Quale modello di processo descrive meglio questa situazione?

- **Modello a sviluppo incrementale**
- Modello a cascata
- Modello prototipale
- Modello a consegna incrementale

Modelli agili

I metodi agili iniziarono ad emgergere alla fine degli anni '90, per cercare di ridurre il tempo di sviluppo del software.
Perchè non ci si può sempre solo ad affidare a modelli plan driven?

- la maggior parte delle volte, i progetti hanno **requisiti non chiari**
- non sempre la quantità di documentazione deve essere elevata
- in alcuni progetti, le variabili e il contesto cambiano rapidamente, quindi è necessario un approccio più flessibile e adattabile

dove non usare modelli agili?

- team numerosi o distanti
- progetti vasti e complessi
- progetti duraturi nel tempo

quando usare modelli agili?

- contesti dove chi arriva prima ha più vantaggi (startup, ecc.)
- non serve documentazione
- progetti dove i requisiti cambiano rapidamente

caratteristiche:

- Documentazione minima
  - ci si concentra sul codice
  - non ci sono specifiche dettagliate
- consegna rapida ed incrementale del software
  - sviluppato in maniera incrementale
  - coinvolgimento degli stakeholder per definire gli incrementi successivi
- Strumenti di sviluppo e automazione
  - strumenti di sviluppo e automazione per ridurre i tempi di sviluppo e testing

Plan-driven	Agile
"Processi dove tutte le attività sono piani cate in anticipo e il loro avanzamento è misurato rispetto a quanto previsto dal piano. Le fasi del processo software sono distinte tra loro e gli	"La piani cazione è incrementale e continua durante lo sviluppo del software. È quindi più facile modi care il processo per adeguarsi alle modi che dei requisiti del cliente o del prodotto. Requisiti, progettazione e

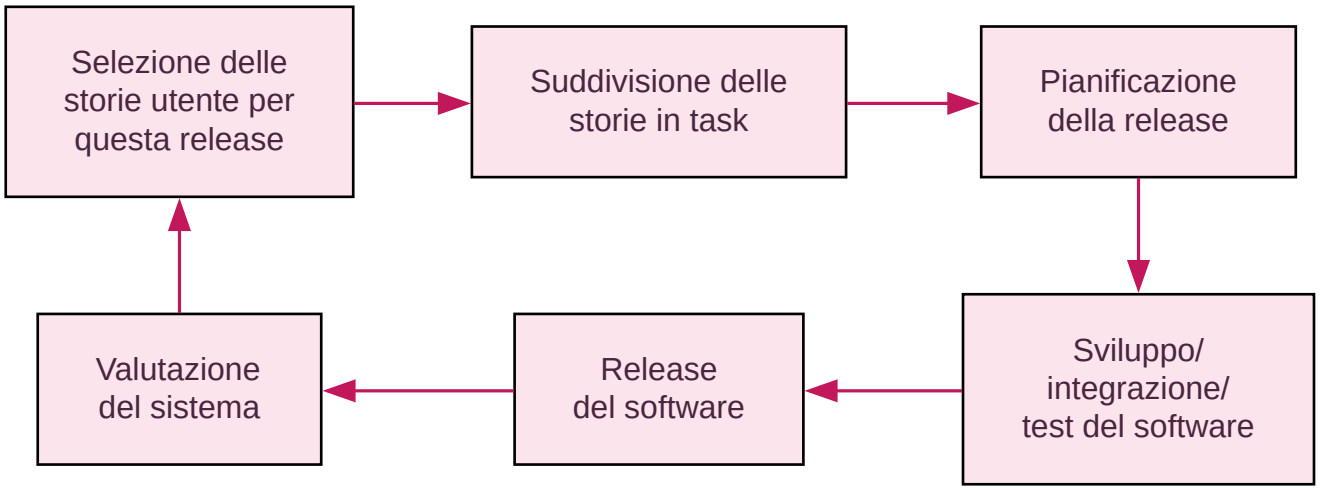
Plan-driven	Agile
output di ciascuna fase sono necessari per la fase successiva"	implementazione avvengono assieme"

slides del prof

principi:

- Coinvolgimento del cliente per tutto il processo di sviluppo
  - velutano e validano l'iterazione
  - forniscono nuovi requisiti
  - assegnano le priorità
- accettare i cambiamenti
- mantenere la semplicità
- sviluppo incrementale
- focus sul team, non processi
  - ogni membro ha il proprio skill set e va assecondato

EXTREME PROGRAMMING



Approccio iterativo estremo: Piccoli e frequenti rilasci al cliente.

- I requisiti sono espressi come storie utente
- Le storie utente sono semplici scenari
- Gli scenari sono divisi in task di sviluppo più semplici
- I programmatori lavorano a coppie
- il test viene sviluppato prima del codice (test driven development)
- Il cliente deve essere coinvolto nello sviluppo
- Il cliente valida la release corrente

principi:

- Coinvolgimento del cliente
  - un rappresentante on-site
- Accettare i cambiamenti
  - appena una task è conculsa viene integrata nel sistema
- Sviluppo incrementale
  - frequenti piccoli rilasci
- Mantenere la semplicità
  - eseguire **refactoring** attivo
- Persone, non processi
  - programmazione in coppia, collaborazione attiva, orari di lavoro flessibili, ecc.
  - si cerca di favorire il benessere del team

checniche che EP ha introdotto:

- Storie utente
- Refactoring
- Sviluppo preceduto dai test
- Pair programming

STORIE UTENTE

- invece di spiegare il requisito con tecnicismi spinti, si spiega con una storiella, in modo che il cliente possa capire
- sono scritte su schede fisiche o virtuali (story cards) che vengono suddivise in task
- i task rilevanti vengono usati per definire la pianificazione delle iterazioni e le stime dei costi
- il compito del cliente è di prendere decisioni sui requisiti con il team
- cliente e team definiscono una priorità

esempio di suddivisione in task:

1. cambia la dose scelta del farmaco prescritto
2. scelta del formulario
3. controllo della dose

- è fondamentale determinare immediatamente le componenti critiche che vanno implementate per prime
- qualora i requisiti cambino, le story card vanno aggiornate di conseguenza

svantaggi:

- non è semplice coprire tutti i requisiti con le storielle

- clienti possono trascurare alcuni scenari o task che ritiene ovvie, ma per gli sviluppatori non lo sono

REFACTORING

anche se è stato introdotto con l'XP , veniva già usato prima senza avere un termine specifico: si cerca di "apparecchiare" il sw in base alla esperienza passata, preparando in anticipo per un possibile cambiamento. è un processo del miglioramente del codice che viene riorganizzato e riscritto per renderlo più mantenibile e comprensibile, senza cambiare le funzionalità.

tradotto in italiano: fai più fatica ora per rendere il sw mantenibile così farai meno fatica nel futuro a fare modifiche
il refactoring è suddiviso in:

- team di sviluppo che crea proattivamente aspetti del sw da migliorare
- il miglioramento può riguadare situazione in cui non vi è un bisogno immediato
- un sw di alta qualità riduce la necessità di documentazione mantentendo facili le modifiche future

esempio di refacotoring necessario:

```
def funzione1():
  a = 1
  b = 2
  c = a + b
  return c


def funzione2():
  t = 1
  m = 2
  z = t + m
  return z
```

entrambe le funzioni eseguono la medesima operazione, potrebbero essere tranquillamente riscritte con una unica funzione (questo si applica anche a parti di funzioni, non solo intere)

pro:

- rende il codice molto più efficente e chiaro, sopratutto se si lavora per sviluppo incrementale
- può essere parzialmente automatizzato

contro:

- non sempre un semplice refactoring è sufficente per ottimizzare una architettura
- "ruba" tempo all'implementazione di altre funzionalità

sviluppo preceduto dai test

scrivo prima i test del codice, in questo modo scrivo un codice che è mirato al passaggio dei test stessi. questo processo rende meno probabile incappare in errori durante lo sviluppo.

i punti principali sono:

- sviluppo test-driven
- automatizzazione dei test
- coinvolgimento del cliente

to-do list dei test drive:

- scrivere i test prima del codice
- automatizzazione dei test
- test scritti come programmi
- possibile eseguire i test mentre si programma in modo da scoprire subito eventuali problemi nel codice
- il codice può considerarsi completo quando tutti i test vengono superati

questo metodo funziona particolarmente bene con i story telling, perchè tramite la narrazione posso esplorare in anticipo i vari possibili scenari e i test necessari

pro:

- il fatto di scrivere un test come un programma ci obbliga ad avere input e output, quindi abbiamo già una interfaccia
- è più chiaro il comportamento
- riduciamo le possibili incomprensioni o ambiguità

contro:

- molto costosa
- i test vanno aggiornati se voglio implementare nuove feature
- i test potrebbero essere incompleti

pair programming

- gruppi di almeno due programmatori che lavorano sullo stesso pezzo di codice
- coppie che cambiano continuamente

pro:

- si condivide conoscenza del codice nel team
- permette processo di revisione con una validazione informale
- si riduce la possibilità di errori
- riduce la possibiltà di turn-over (se un programmatore se ne va, rimane l'altro elemento della coppia che sa come funziona)

contro:

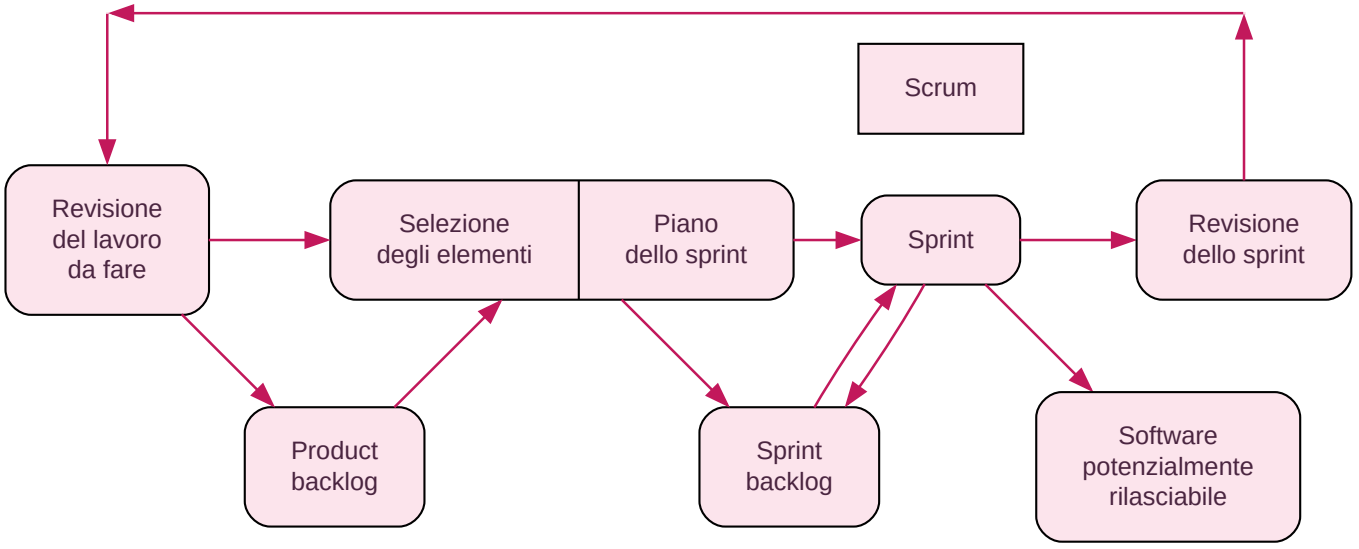
- se le coppie sono mal bilanciate posso ridurre la produttività

Gestione agile della progettazione

nota: questa sezione è estremamente probabile venga chiesta all'esame

SCRUM

L'esigenza nasce da una situazione dove i manager notano che gli sviluppatori stanno producendo, ma non è ben chiaro quando il sw sarà pronto:
il modello agile si scontrò con la visibilità del processo (sopratutto a causa della pianificazione infomale) e i manager non erano in grado di controllare il processo e la gestione del tempo e budget.



- ogni **sprint** parte dal product backlog (lista degli elementi del progetto), backlog che parte spesso dai requisiti (ma non necessariamente) e va ordinato in una lista di cose da fare (una lista, un documento formale, storie utente, ecc.)
- all'inizio di ogni ciclo ci sarà una entità (**product owner** - un cliente, un product manager, una persona incaricata da noi, etc..) che definisce le priorità da sviluppare in quel

determinato sprint
    - tutti i membri del team contribuiscono alla pianificazione dello sprint, stimando il tempo necessario (**velocity**) per completare le attività e definendo gli obiettivi da raggiungere, tenendo conto anche degli sprint passati
- sprint:
    - il team scegli chi deve lavorare su quali attività
    - il team di sviluppo ha dimensioni contenute (10 persone circa)
    - dura 2-4 settimane
- scrum master:
    - runione giornaliera per ispezionare lo stato del progetto per aggiornare lo sprint backlog
    - NON è il project manager, ma un facilitatore del processo
    - breve incontro faccia a faccia con il team di sviluppo
    - Lo ScrumMaster ha la responsabilità di garantire che il processo Scrum sia seguito e di guidare il team nell'uso efficente di Scrum
- incremento:
    - ogni incremento deve essere un prodotto funzionante e potenzialmente rilasciabile
    - non sempre è completo
    - può essere rilasciato prima della fine dello sprint
- revisione dello sprint:
    - riunione di verifica (sprint review) che coinvolge tutti gli elementi del team
    - ci si concentra su:
        - **sprint review**: input sul prodotto e sul suo stato per la revisione del product backlog
        - **sprint retrospective**: il team riflette sul processo e identifica aree di miglioramento per il prossimo sprint

Tipologie di requisiti

definizione dei requisiti

bisogna definire:

- esigenze del cliente
- vincoli del sistema (se un prodotto funziona ma non rispetta i vincoli, non è un buon prodotto)

Descrizione di qualcosa che il sistema dovrà fare oppure di una proprietà o vincolo operativo che si desidera per il sistema, definiti da:

- formulazione astratta di alto livello
- specifica dettagliata (es: linguaggio matematico)

la prcedente definizione è generica, perchè un requisito ha un amplio ventaglio di scopi, ma L'**ingegneria dei requisiti** è il processo di ricerca, analisi, documentazione e verifica dei requisiti

è fondamentale per prevenire e mitigare gli errori, perchè più tardi si scopre un errore, più costoso sarà correggerlo.
requisiti utente:

- frasi in linguaggio naturale relative alle funzionalità che il sistema deve fornire e i suoi vincoli operativi
- generalmente scritti in alto livello
- descritti solitamente usando un linguaggio naturale e diagrammi **comprensibili a tutti**

requisiti di sistema:

- descizione dettagliata delle funzionalità del sistema e dei vincoli operativi
- definisce cosa deve essere sviluppato
- può essere parte del contratto tra cliente e sviluppatore

requisiti funzionali e non funzionali

requisiti funzionali:

- descrivono ciò che il sistema dovrebbe fare

requisiti non funzionali (tutto il resto):

- Vincoli sulle funzionalità offerte dal sistema o vincoli sul processo di sviluppo
- Includono i requisiti di qualità

requisiti non funzionali

- I requisiti non funzionali possono essere più critici dei requisiti funzionali
- Se non sono soddisfatti, il sistema potrebbe essere inutilizzabile
- difficile da identificare quali componenti del sistema implementano specifici requisiti non funzionali
- possono influire sulla intera architettura del sistema
- un requisito non funzionale può generare una serie di requisiti funzionali

requisiti del prodotto:

- Derivano dalle caratteristiche richieste al software
- Specificano il comportamento del prodotto (usabilità, efficienza, prestazioni)

requisiti organizzativi:

- Derivano da politiche e procedure dell’organizzazione che sviluppa il software e del cliente

requisiti esterni:

- Tutti i requisiti che derivano da fattori esterni al sistema e al suo processo di sviluppo

verificabilità dei requisiti

- I requisiti non sempre sono facili da verificare
- Il cliente li specifica come obiettivi generici/vaghi (possono causare problemi agli sviluppatori per la loro interpretazione)
- I requisiti devono essere vericabili

requisiti di dominio:

- le specifiche di dominio potrebbero non essere note agli sviluppatori
- talvolta un esperto del dominio può non fornire requisiti completi, perchè ritiene che alcune informazioni siano ovvie, ma non lo sono per gli sviluppatori
- L’ingegnere del software potrebbe non conoscere bene il dominio